Project RELAY · the dispatch desk

Recommend the right rider, assign with a reason, and never lose a job that stalls.

MERN, plain JavaScript
React 18 + Vite · Express 5 + Node 20 · MongoDB Atlas (transactions)

1 The problem and the solution

Today: dispatchers check availability, call riders and update spreadsheets. When a rider doesn't answer or nobody fits, an order sits idle and nobody notices.

RELAY: one desk where every job has a clear stage and owner.

- Ranks the riders who *can* take a job, with the reasons.
- The dispatcher assigns the top rider, or another with a recorded reason.
- The rider must confirm within **2 hours** (configurable). Declined or unanswered jobs move to the **Manual Queue** by themselves, with capacity released.
- Every action is in the job's history: who, when, why.

2 The dispatcher's workflow

1. **Jobs dashboard:** counts per stage and what needs attention: manual queue, confirmations due soon, past deadline.
2. **Open a job:** ranked riders with reasons; who is not eligible, and why.
3. **Assign** the recommended rider, or **Assign instead** with a reason.
4. **Rider confirms** in the app (or by phone, recorded).
5. **Picked up → Delivered.** Invalid moves are blocked by the server.

> **Exceptions** (decline, no answer, given back, taken back, nobody eligible) → **Manual Queue** with the reason → reassign or cancel.

3 Ranking riders and auto-queueing

Eligible = available, covers both zones, has room for the load, below the job limit, hasn't declined or given back this job.

Score 0–100 from four parts; urgent jobs weigh speed and deadline more:

Priority	Travel	Workload	Deadline	Capacity
Low / Normal	40	25	20	15
High / Urgent	55	10	25	10

No answer on this job earlier: **−20**. Each rider shows reasons and why they rank above the next.

Timeout: every 15 s, overdue attempts expire in **one transaction**: attempt → EXPIRED, capacity released, job → Manual Queue with the reason, history written. Late answers are refused; a restart loses nothing.

4 Architecture and data model

Web app (React) ⇄ **API** (Express, Node) ⇄ **Database** (MongoDB Atlas)

API: thin routes → services (one transaction per workflow change) → pure rule functions, plus a 15 s timeout check.

- `jobs`: one per order (**unique** order ref).
- `assignments`: one per attempt, all kept; **unique partial index** = one live attempt per job.
- `jobevents`: append-only history of every action.
- `riders`: capacity reserved by **one conditional update**, never over-booked.
- `users` (OTP login, roles) · `settings` (timeout).

5 Trade-off: polling, not a job queue

Timeouts are found by a 15-second database poll, not by a Redis/BullMQ delayed job per assignment.

+ No new infrastructure; all state stays in MongoDB.
+ Survives restarts: the check also runs at startup.
+ Safe with several servers: each expiry is a conditional update, so it happens once.
− A job can move up to 15 s late (fine against 2 hours).
− Every server polls. At scale: one worker or a delayed-job queue.

6 With another week

1. **Autonomous dispatch:** offer each job to the top rider automatically and move down the list on decline or no answer; alert when an urgent job waits in the queue.
2. **Live updates and rider alerts:** MongoDB change streams over WebSockets instead of polling; push or SMS for new offers.
3. **Safer manual control:** urgent override (re-offer to a rider who declined, or call in an off-duty rider) with a recorded reason; history of rider edits; idempotency keys on every POST.

Then: tune the weights from real deliveries, real locations with a geospatial index.

Concurrency: one transaction per workflow change · unique indexes (order ref; one live attempt per job) · conditional updates on job status + version, rider capacity and attempt status. Two clicks at once: one wins, the other gets a clear 409.

Tests: unit tests for the rules; API + database tests for the 8 required scenarios and the races (double assign, last capacity, accept vs timeout). Demo controls move the clock to show the 2-hour timeout.